

GitHub Organization Profile: NEXUS-R

Personal agent runtime. Progressive cost reduction through verified workflow reuse.

Public Bio (160 characters)

NEXUS-R: A local-first AI agent runtime that makes task execution progressively cheaper through verified workflow caching. Not AGI. Just infrastructure.

Organization Description

NEXUS-R builds the execution layer for autonomous AI agents — the runtime that makes agents cheaper to operate over time, not more expensive.

We are a systems-engineering project focused on:

- **Operational correctness** over demo polish
- **Failure recovery** over feature count
- **Cost transparency** over black-box magic
- **Local-first privacy** over cloud dependency

Our core mechanism, Execution Trace Distillation (ETD), transforms successful agent executions into reusable, parameterized workflows with quantified reliability guarantees. This is not caching. It is verified knowledge accumulation.

Current state: Phase 1.5 complete. Real Ollama integration validated. 100 concurrent tasks, zero failures. Event-sourced architecture operational.

Next: Phase 2 — ETD pipeline, full 4-tier routing, cost dashboard.

Pinned Repositories

Repository	Description	Language	Stars	Status
nexus-r	Core runtime —agent execution, routing, sandbox, event sourcing	Python	—	Active
nexus-r-mcp-tools	Community MCP tool servers for NEXUS-R integration	Python/TypeScript	—	Planned
nexus-r-etd-hub	Community workflow exchange — privacy-stripped ETD sharing	Python	—	Phase 4

Labels and Topics

```
# Primary
topics: ai-agent-runtime, local-first-ai, event-sourcing, workflow-automation, llm-infrastructure

# Secondary
topics: mcp-protocol, litellm, cost-optimization, agent-native-development, operational-telemetry

# Not used
# agi, artificial-general-intelligence, superintelligence, revolutionary-ai, next-gen-cognition
```

README for Organization Page

```
# NEXUS-R Labs

We build the runtime layer for autonomous AI agents.

## What We Do

Current AI agent architectures treat every execution as a novel problem, routing each request to the
↳ most expensive model regardless of history. This is economically unsustainable at scale.

NEXUS-R inverts this: every successful execution makes the next one faster and cheaper, through a
↳ verified cache of reusable execution plans called Execution Trace Distillation (ETD).

## Principles

1. Operational honesty — We report what works, what doesn't, and what's mocked.
2. Local-first — Your data stays on your machine. Cloud is optional, not default.
3. Deny-first security — Default is block. Permission is earned through verification.
4. Specification-driven — Interfaces before implementation. Tests before integration.

## Repositories

- [nexus-r] (https://github.com/nexus-r/nexus-r) — Core runtime (Python, Phase 1.5)
- [nexus-r-specs] (https://github.com/nexus-r/nexus-r-specs) — Public specification documents
- [nexus-r-benchmarks] (https://github.com/nexus-r/nexus-r-benchmarks) — Reproducible performance
  ↳ data

## Getting Involved

- Read the [architecture] (https://github.com/nexus-r/nexus-r/blob/main/ARCHITECTURE.md)
- Review [runtime truthfulness] (https://github.com/nexus-r/nexus-r/blob/main/
  ↳ runtime_truthfulness_report_phase1_5.md)
- Check [open issues] (https://github.com/nexus-r/nexus-r/issues) tagged `good-first-issue`

## Contact

- Engineering discussions: [GitHub Discussions] (https://github.com/nexus-r/nexus-r/discussions)
```

```
- Security reports: security@nexus-r.dev
- General: hello@nexus-r.dev
```

Social Media Presence (Optional)

Platform	Handle	Content Focus
X/Twitter	@nexus_r_dev	Engineering updates, benchmark results, architecture decisions
LinkedIn	NEXUS-R	Company page for hiring, investor updates
Bluesky	@nexus-r.dev	Long-form technical posts, failure analyses

Content rules:

- Share benchmarks with methodology, not just numbers
- Discuss failures openly (“ETD generalization dropped to 85% — here’s why”)
- No hype threads. No “revolutionary” claims. No AGI speculation.

Investor / Demo Narrative

For technical investors:

NEXUS-R is building the execution infrastructure layer for the emerging autonomous agent ecosystem. Our core mechanism, Execution Trace Distillation, creates structural switching costs through accumulated workflow knowledge — not contractual lock-in. We are currently at Phase 1.5 with validated local runtime, event sourcing, and concurrency handling. Phase 2 will validate the core economic thesis: >40% cost reduction on repeated tasks through verified workflow caching.

Demo script:

1. `nexus run "list all python files"` — shows T1, local model, \$0 cost, 1.2s
2. `nexus run "explain this complex error"` — shows T3, fallback to BYOK, cost logged
3. `nexus history` — shows full audit trail with event IDs
4. `nexus cost` — shows cumulative breakdown, local vs cloud
5. Stress test: run 20 tasks concurrently, show zero failures
6. Show `runtime_truthfulness_report` — honest A/B/C/D classification

What NOT to say:

- “This will revolutionize AI”
- “We’re building AGI infrastructure”
- “Unlimited scalability”
- “Zero latency”

What TO say:

- “Validated on 100 concurrent tasks with zero failures”
- “Cost reduction is measurable, not theoretical”
- “Every subsystem has an honest truthfulness rating”
- “We kill features that don’t prove economic value”

Contribution Velocity Signals

For GitHub profile health (investors/contributors check these):

Signal	Target	How
Commit frequency	3-5/week	Agent-native development produces steady commits
Issue response time	<48 hours	Meta-Agent (you) triages quickly
PR review quality	Spec + tests mandatory	No code without adversarial tests
Security transparency	Public audit reports	security_audit_phase*.md in repo
Performance transparency	Public benchmarks	benchmark_report_phase*.md in repo
Honest limitations	known_limitations.md updated	No pretending unfinished is done

The repository should feel like serious infrastructure, not a science project.